

ANGULAR FOR JAVA BACKEND ENGINEERS

Java 工程師的 Angular API 開發 入門

DI、Service、HttpClient、Proxy、MSW 與 RxJS



核心概念

DI 把 object creation 從 business code 中移出去。

DI (Dependency Injection , 依賴注入) 就是「不要自己建立需要的物件，而是向 Angular 要。」

沒有 DI

```
class OrderComponent {  
  apiService = new ApiService();  
}
```

class 自己決定 implementation 與 lifecycle ; 每個地方都可能建立不同的實例。測試與替換都會變得更困難。

使用 DI

```
class OrderComponent {  
  constructor(private apiService: ApiService) {}  
}  
  
// 或使用新版 inject() 寫法 (Angular 14+)  
class OrderComponentWithInject {  
  private apiService = inject(ApiService);  
}
```

class 只宣告自己需要什麼。
Angular Injector 會負責建立並提供對應的 instance。

先用熟悉的 Spring DI 理解 Angular DI。

MANAGED OBJECT

Spring Bean



Angular Service

由 framework 管理建立與生命週期。

CONTAINER

ApplicationContext



Angular Injector

找到 dependency 並提供對應 instance。

INJECTION

Constructor Injection



constructor / inject()

Class 只宣告需要什麼，不自己 new。

先記住一句話：Angular DI 的使用方式，和你在 Spring 裡做的事情非常接近。

ANGULAR SERVICE

需要跨 component 重用邏輯時， 就該抽成 Service。

狀態共享

登入者資料、權限角色、目前案件資料，不應散落在每個 component。

商業流程

權限判斷、格式轉換，適合放在 Service 裡集中測試。

外部資源

API、localStorage、sessionStorage 這類外部資源，由 Service 統一包起來。

Angular Service 負責封裝可重用的業務邏輯、資料存取（API 呼叫）、狀態管理或工具方法，避免這些邏輯散落在各個 Component 中。

API SERVICE

把打 API 的方法集中， component 只 呼叫方法。

- 統一 URL、headers、錯誤處理與 response mapping。
- component 不需要知道 endpoint 細節，只依賴 method contract。
- 測試時可以 mock service，不必真的打 HTTP。

集中 API 呼叫

```
@Injectable({ providedIn: 'root' })
export class ApiService {
  private http = inject(HttpClient);

  baseRequest: BaseRequest<unknown> = {
    MWHEADER: {
      TXNSEQ: '12345678901234'
    }
  };

  getToken(tranrq: GetTokenTranrq) {
    const body: BaseRequest<GetTokenTranrq> =
      Object.assign({ TRANRQ: tranrq }, this.baseRequest);

    return this.http.post<BaseResponse<GetTokenRes>>(
      '/v1/saml/getToken',
      body
    );
  }
}
```

Request 固定帶 MWHEADER；業務參數放在 TRANRQ，response 由 TRANRS 承接。

固定外層由 API Service 處理，業務程式只使用 TRANRQ / TRANRS。

固定 ENVELOPE

```
interface MwHeaderReq {
  TXNSEQ: string;
}

interface BaseRequest<T> {
  MWHEADER: MwHeaderReq;
  TRANRQ?: T;
}

interface MwHeaderRes {
  RETURNCODE: string;
  RETURNDESC: string;
  TXNSEQ: string;
}

interface BaseResponse<T> {
  MWHEADER: MwHeaderRes;
  TRANRS: T;
}
```

外層欄位固定；每支 API 只有 TRANRQ / TRANRS 的泛型內容不同。

COMPONENT

送出業務資料・接收 User

REQUEST ↓ RESPONSE ↑

API SERVICE

包裝 MWHEADER・檢查 RETURNCODE・取出 TRANRS

ENVELOPE ↓ ENVELOPE ↑

COMPANY API

接收 Request・回傳 Response Envelope

RESPONSE MAPPING

interface 描述格式；model 建立穩定的前端資料。

以前：INTERFACE

只在 compile time 檢查型別。API 少給欄位時，runtime 仍可能拿到 undefined。

現在：MODEL

在 API 邊界補預設值、固定轉換資料，component 只依賴穩定的 View Model。

Model 的預設值是防止畫面因缺少欄位而壞掉；但必要欄位仍不能默默補空值，應該驗證或回報錯誤。

API SERVICE 轉換

```
interface GetTokenTranrq {  
  USER_ID: string;  
};  
  
class GetTokenRes {  
  TOKEN = '';  
  
  constructor(res?: any) {  
    this.TOKEN = res?.TOKEN || '';  
  }  
}  
  
getToken(tranrq: GetTokenTranrq) {  
  const body: BaseRequest<GetTokenTranrq> =  
    Object.assign({ TRANRQ: tranrq }, this.baseRequest);  
  
  return this.http  
    .post<BaseResponse<GetTokenRes>>('/v1/saml/getToken', body)  
    .pipe(map(res => new GetTokenRes(res.TRANRS)));  
}
```

`post<BaseResponse<GetTokenRes>>()` 只是告訴 TypeScript 回傳型別，不會自動執行 constructor。所以接到資料後，要明確從 TRANRS 建立 model，讓預設值與轉換邏輯真的生效。

HTTPCLIENT

公司 API 統一使用 HttpClient.post<T>()。

公司使用方式

```
const body: BaseRequest<GetTokenTranrq> =  
  Object.assign(  
    { TRANRQ: { USER_ID: userId } },  
    this.baseRequest  
  );  
  
this.http.post<BaseResponse<GetTokenRes>>(  
  '/v1/saml/getToken',  
  body  
);
```

關鍵心智模型

- POST 的第二個參數是 BaseRequest : MWHEADER 加上 TRANRQ。
- 透過 BaseResponse<T> 描述 MWHEADER 與 TRANRS。
- Interceptor 可集中處理 token、logging、錯誤。

DEVELOPMENT PROXY

localhost 收到 service prefix，再由 dev server 轉送到 SIT。



Proxy 先用 `/user` 選擇 target，再移除 prefix 後轉送。

Proxy 只作用於 `ng serve`；部署到測試機後改由 Nginx 設定。

API path 加上 service prefix，組成前端呼叫 URL。

MAPPING

```
export const urlMapping = {  
  '/v1/user/xxx': 'user'  
};  
  
const urlPrefix = urlMapping[path];  
const url =  
  `${urlPrefix}${path}`;
```

01 · FRONTEND

/user/v1/user/xxx

proxy 以 /user 選擇 User API，並移除 prefix

02 · TARGET

http://sit-user-api.example.com/v1/user/xxx

Prefix 是前端 routing key，不是 Backend API 原始 path 的一部分。

PROXY.CONF.JSON

用 service prefix 選擇 target，轉送前移除 prefix。

PROXY.CONF.JSON

```
{
  "/user/**": {
    "target": "http://sit-user-api.example.com",
    "changeOrigin": true,
    "secure": false,
    "pathRewrite": {
      "^/user": ""
    }
  }
}
```

`/user/v1/user/xxx` 轉送後會變成 target 的 `/v1/user/xxx`。

ANGULAR.JSON

```
"serve": {
  "builder": "@angular/build:dev-server",
  "options": {
    "proxyConfig": "src/proxy.conf.json"
  }
}
```

- 啟動時執行 `ng serve`。
- 修改 proxy 設定後要重啟 dev server。
- 前端只寫 `/user/...` 這種相對路徑，不在程式碼寫死 SIT host。

proxy.conf.json 設定

PROXY.CONF.JSON

```
{
  "/user/**": {
    "target": "http://sit-user-api.example.com",
    "changeOrigin": true,
    "secure": false,
    "pathRewrite": {
      "^/user": ""
    }
  }
}
```

範例：前端打 `/user/v1/user/xxx`，最後送到 SIT 的 `http://sit-user-api.example.com/v1/user/xxx`。

`/user/**`

只要 request path 以 `/user/` 開頭，就套用這組 proxy 規則。

`target`

真正要轉送到的後端 SIT domain；這裡是 User API 的測試機。

`changeOrigin`

轉送時把 Host header 改成 target 的 host，讓後端看起來像是直接收到該 domain 的 request。

`secure`

是否驗證 HTTPS target 的憑證；公司 SIT 若是自簽憑證常設 false。若 target 是 HTTP，這個設定基本上不會影響連線。

`pathRewrite`

轉送前先改寫 URL path，通常用來移除前端自己加上的 service prefix。

`"^/user": ""`

把開頭的 `/user` 替換成空字串，所以 `/user/v1/user/xxx` 會變成 `/v1/user/xxx`。

同一套 prefix routing，依環境由不同 server 執行。

本機開發

proxy.conf.json

```
{
  "/user/**": {
    "target": "http://sit-user-api.example.com",
    "secure": false,
    "pathRewrite": { "^/user": "" }
  }
}
```

執行者：Angular dev server

部署到測試機

nginx/default.conf

```
location /user/ {
  proxy_pass http://sit-user-api.example.com/;
}
```

執行者：測試機上的 Nginx

新增 service prefix 或 target domain 時，必須同步更新 proxy.conf.json 與 nginx/default.conf。

MOCK SERVICE WORKER

API 還沒好時，MSW 在 Browser 攔截 request 並回傳假資料。



準備 worker，並只在開發模式啟用 mocking。

初始化

```
npx msw init public/ --save

// src/mocks/browser.ts
import { setupWorker } from 'msw/browser';
import { handlers } from './handlers';

export const worker =
  setupWorker(...handlers);
```

CLI 會產生 public/mockServiceWorker.js，Browser 透過它攔截 request。

MAIN.TS

```
async function enableMocking() {
  if (!environment.isMock) return;

  const { worker } =
    await import('./mocks/browser');

  await worker.start({
    //...
  });

  enableMocking().then(() =>
    bootstrapApplication(AppComponent)
  );
}
```

必須 await worker.start()，避免 App 初始 API 比 worker 更早送出。

Handler 負責攔截；JSON 檔負責提供可維護的假資料。

SRC/MOCKS/HANDLERS.TS

```
import { http, HttpResponse } from 'msw';
import userData from './data/user-data.json';

export const handlers = [
  http.post('/user/v1/user/data', async () => {
    return HttpResponse.json(userData);
  })
];
```

Handler URL 要和前端實際呼叫的 URL 完全對得上。

SRC/MOCKS/DATA/USER-DATA.JSON

```
{
  "MWHEADER": {
    "TXNSEQ": "2985070176776",
    "RETURNCODE": "0000",
    "RETURNDISC": "交易成功"
  },
  "TRANRS": {
    "DEP_NO": "21A0000",
    "DEP_NAME": "風險管理部"
  }
}
```

- 假資料與 handler 分開管理。
- Response 外層維持公司固定格式。
- 需要時可準備多組情境 JSON。

RXJS BASICS

Observable 是未來資料的 stream。

可以把它想成：資料可能現在、稍後、很多次，或永遠不會到。RxJS 提供 operators 來轉換與組合。

Observable

資料來源

**pipe()**串接
operators**subscribe()**

開始接收結果

```
this.apiService.getUsers().pipe(  
  map(users => users.filter(u => u.active)),  
  catchError(() => of([]))  
)
```

Observable 描述資料來源， subscribe 才開始接收。

在 Angular 裡，HTTP、表單、router events 都常用 Observable 表達非同步資料。

- Observable 是「可被訂閱的資料流」，不一定立刻產生資料。
- subscribe 是消費端，負責接收 next、error、complete。

```
this.apiService.getUsers().subscribe({  
  next: users => this.users = users,  
  error: err => this.message = err.message,  
  complete: () => this.loading = false,  
});
```

SUBJECT

Subject 同時是 producer，也是 Observable。

它能 NEXT

Service 可以透過 Subject 主動推送事件，例如使用者登入、modal 開關、跨 component 通知。

```
private saved$ = new Subject<void>();

notifySaved() {
  this.saved$.next();
}
```

也能 SUBSCRIBE

其他 component 可以訂閱同一個 Subject。它像一條事件 bus，但仍要注意生命週期與退訂。

```
this.saved$.subscribe(() => {
  this.reload();
});
```

SUBJECT VS BEHAVIORSUBJECT

差別在於：BehaviorSubject 會記住目前值。

SUBJECT

只有 subscribe 之後的新事件會收到。適合表達一次性事件。

```
const click$ = new Subject<string>();
click$.next('A');
click$.subscribe((value) => {
  console.log(value);
});
click$.next('B'); // only B
```

BEHAVIORSUBJECT

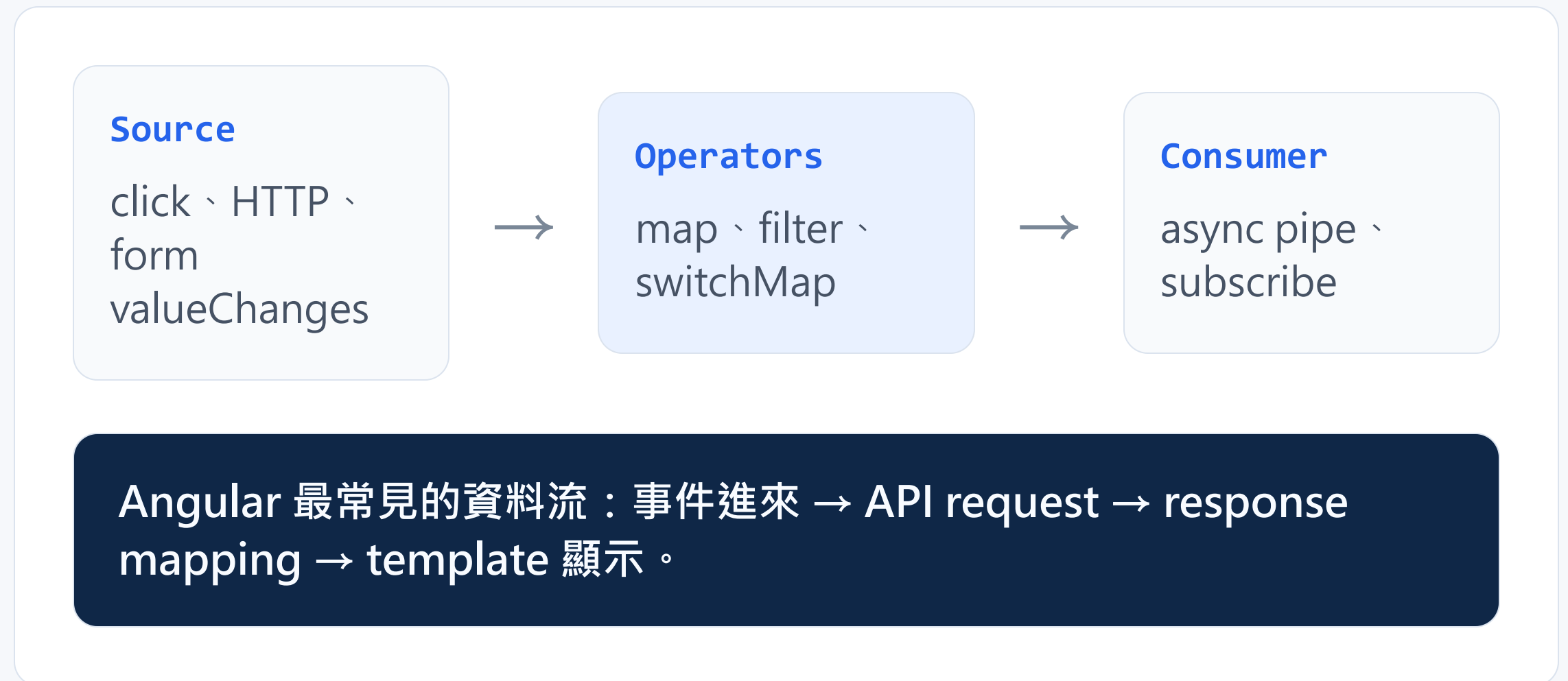
需要初始值；新 subscriber 會立刻拿到最新值。適合表達狀態。

```
const user$ = new BehaviorSubject<User | null>(null);
user$.next(currentUser);
user$.subscribe((value) => {
  console.log(value); // gets currentUser
});
```

資料流概念

不要想成變數
被改；想成資
料沿著
stream 往下
游走。

來源送出值，operators 轉換
值，最後由 template 或
component 消費結果。



常用 RXJS

先熟這幾組，就能讀懂大多數 Angular 專案。

轉換

map

filter

tap

整理資料、排除不需要的值、插入 side effect。

切換 API

switchMap

條件改變時取消舊 request，只保留最新一次結果。

錯誤與重試

catchError

retry

finalize

把失敗路徑收斂在 API Service。

組合與快取

forkJoin

debounceTime

shareReplay

等待多個 API 都完成、降低頻率、重用結果。